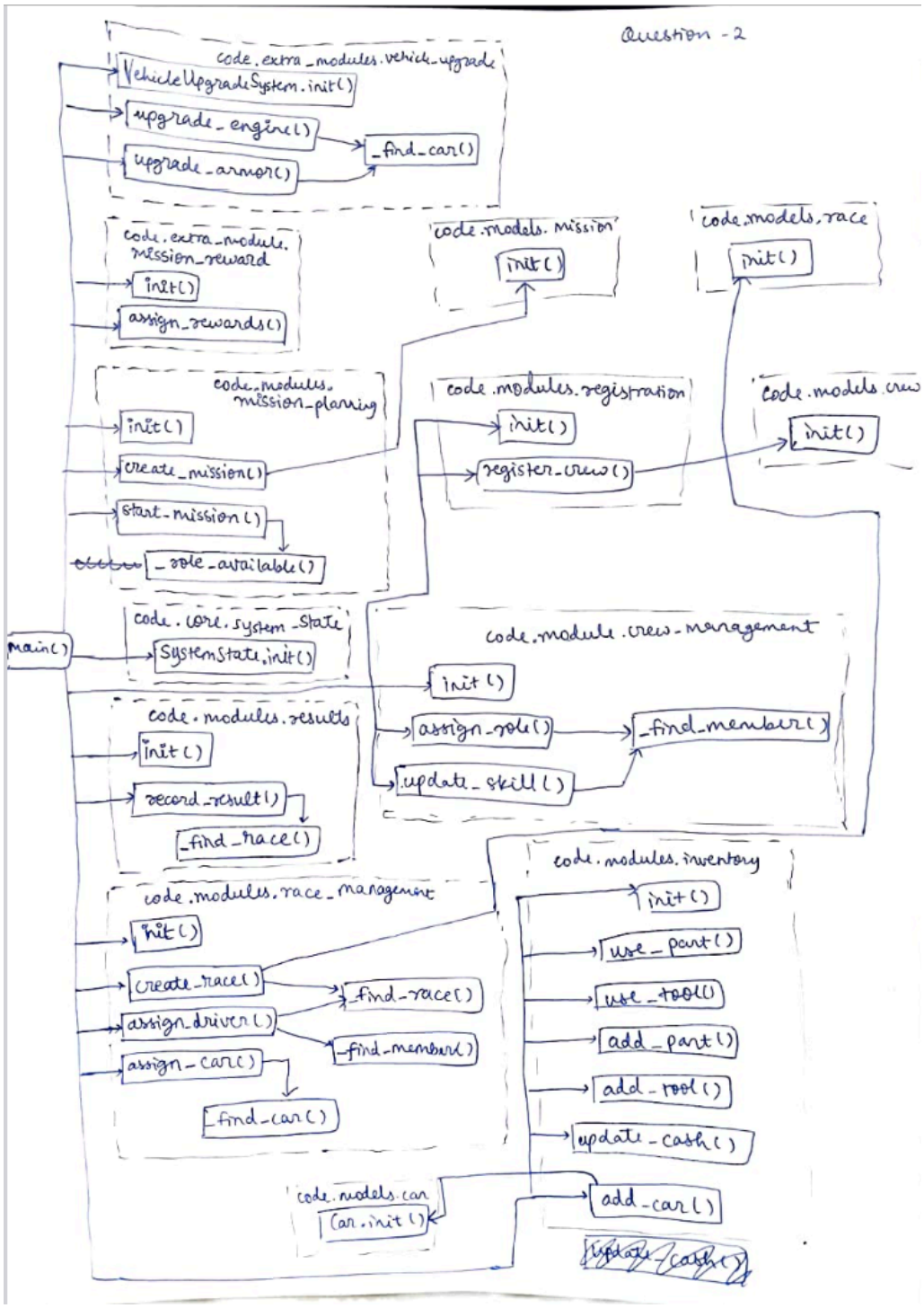
DASS Assignment 2

Roll Number: 2024101096

Answer 2.1



Answer 2.2

Test Case 1

Scenario: Register crew and assign role
Modules Involved:

1. RegistrationModule
2. CrewManagementModule

Steps:

1. Register a crew member "Alex"
2. Assign role "driver" to Alex

Expected Result:

➔ Crew member is successfully registered
➔ Role "driver" is assigned correctly

Actual Result:

➔ Crew member "Alex" registered successfully
➔ Role assigned as "driver"

Errors / Issues Found: No errors found
Explanation: This test ensures that **Crew Management depends correctly on Registration**, meaning roles can only be assigned to existing crew members.

Test Case 2

Scenario: Add car and use in race
Modules Involved:

1. InventoryModule
2. RaceManagementModule

Steps:

1. Add car "CAR1" to inventory
2. Create race
3. Assign car "CAR1" to race
4. **Expected Result:**

➔ Car should be successfully assigned to race

Actual Result:

➔ Car assigned successfully

Errors / Issues Found: No errors found
Explanation: This test ensures that **cars from inventory are correctly used in race management**, validating cross-module data flow.

Test Case 3

Scenario: Assign unusable car to race
Modules Involved:

1. InventoryModule
2. RaceManagementModule

Steps:

1. Add car with condition = 0
2. Create race
3. Attempt to assign this car

Expected Result:

➔ System should reject the car

Actual Result:

➔ Error returned: "Car is not usable"

Errors / Issues Found: No errors found
Explanation: This test ensures that **race module validates car condition**, preventing invalid race setups.

Test Case 4

Scenario: Complete race and update inventory
Modules Involved:

1. RaceManagementModule
2. ResultsModule
3. InventoryModule

Steps:

1. Create race
2. Assign driver
3. Record result as "win"

Expected Result:

➔ Race result recorded
➔ Cash increases by reward amount

Actual Result:

➔ Result recorded
➔ Cash updated correctly

Errors / Issues Found: No errors found
Explanation: This test validates that **race results correctly impact inventory**, ensuring reward flow is working.

Test Case 5

Scenario: Assign a driver with an invalid role (non-driver) to a race

Modules Involved:

1. RaceManagementModule
2. CrewManagementModule

Steps:

1. Register a crew member "Bob"
2. Assign role "mechanic" to Bob
3. Create a race
4. Attempt to assign Bob as driver

Expected Result:

- ➔ System should reject the assignment
- ➔ Error message should indicate only drivers are allowed

Actual Result:

- ➔ System returned error: "Only drivers can be assigned"

Errors / Issues Found: No errors found

Explanation: This test ensures that **only crew members with the driver role can participate in races**, preventing invalid race configurations.

Test Case 6

Scenario: Assign a driver to a race without registering the crew member

Modules Involved:

1. RaceManagementModule

Steps:

1. Create a race
2. Attempt to assign a non-existent crew member as driver

Expected Result:

- ➔ System should return error: "Crew member not found"

Actual Result:

- ➔ Error returned correctly

Errors / Issues Found: No errors found

Explanation: This test ensures that **race participation strictly depends on prior crew registration**, maintaining system consistency.

Test Case 7

Scenario: Record race result without assigning a driver

Modules Involved:

1. RaceManagementModule
2. ResultsModule

Steps:

1. Create a race
2. Attempt to record result without assigning a driver

Expected Result:

- ➔ System should not allow result recording
- ➔ Error message should indicate missing driver

Actual Result:

- ➔ Error returned: "No driver assigned"

Errors / Issues Found: No errors found

Explanation: This test ensures that **a race must be properly set up before recording results**, preventing incomplete operations.

Test Case 8

Scenario: Record result for a non-existent race

Modules Involved:

1. ResultsModule

Steps:

1. Attempt to record result for a race ID that does not exist

Expected Result:

➔ System should return error: "Race not found"

Actual Result:

➔ Error returned correctly

Errors / Issues Found: No errors found

Explanation: This test ensures that **the system does not allow operations on invalid or missing race data**, improving robustness.

Test Case 9

Scenario: Complete race with result "lose"

Modules Involved:

1. RaceManagementModule
2. ResultsModule
3. InventoryModule

Steps:

1. Register a crew member "Alex"
2. Assign role "driver"
3. Add a car
4. Create a race
5. Assign driver and car
6. Record result as "lose"

Expected Result:

➔ Cash should not increase
➔ Rankings should remain unchanged

Actual Result:

- ➔ Cash remained unchanged
- ➔ Rankings not updated

Errors / Issues Found: No errors found

Explanation: This test ensures that **rewards are only given for winning scenarios**, maintaining correct business logic.

Test Case 10

Scenario: Complete multiple races and verify cumulative rewards

Modules Involved:

1. RaceManagementModule
2. ResultsModule
3. InventoryModule

Steps:

1. Register a driver
2. Add a car
3. Create and complete two races with result "win"

Expected Result:

- ➔ Cash should increase cumulatively
- ➔ Rankings should increase cumulatively

Actual Result:

- ➔ Cash updated correctly after both races
- ➔ Rankings increased as expected

Errors / Issues Found: No errors found

Explanation: This test ensures that **system state updates accumulate correctly over multiple operations**, maintaining consistency.

Test Case 11

Scenario: Assign a car to a non-existent race

Modules Involved:

1. InventoryModule
2. RaceManagementModule

Steps:

1. Add a car
2. Attempt to assign it to a non-existent race

Expected Result:

➔ System should return error: "Race not found"

Actual Result:

➔ Error returned correctly

Errors / Issues Found: No errors found

Explanation: This test ensures that **cars are only assigned to valid races**, preventing invalid operations.

Test Case 12

Scenario: Assign a non-existent car to a race

Modules Involved:

1. RaceManagementModule

Steps:

1. Create a race
2. Attempt to assign a car that does not exist

Expected Result:

➔ System should return error: "Car not found"

Actual Result:

➔ Error returned correctly

Errors / Issues Found: No errors found

Explanation: This test ensures that **only cars present in inventory can be used in races**, maintaining data integrity.

Test Case 13

Scenario: Start mission with required role available

Modules Involved:

1. RegistrationModule
2. CrewManagementModule
3. MissionPlanningModule

Steps:

1. Register a crew member
2. Assign required role (mechanic)
3. Create a mission requiring that role
4. Start the mission

Expected Result:

➔ Mission should start successfully
➔ Mission status should be updated

Actual Result:

➔ Mission completed successfully

Errors / Issues Found: No errors found

Explanation: This test ensures that **mission execution depends correctly on crew role availability**.

Test Case 14

Scenario: Start mission with missing required role

Modules Involved:

1. MissionPlanningModule

Steps:

1. Create a mission requiring a role
2. Attempt to start mission without available crew

Expected Result:

➔ System should return error

Actual Result:

➔ Error returned correctly

Errors / Issues Found: No errors found

Explanation: This test ensures that **missions cannot proceed without required roles**, enforcing system constraints.

Test Case 15

Scenario: Assign rewards for completed delivery mission

Modules Involved:

1. MissionPlanningModule
2. MissionRewardsSystem
3. InventoryModule

Steps:

1. Create a delivery mission
2. Mark mission as completed
3. Assign rewards

Expected Result:

➔ Cash should increase

Actual Result:

➔ Cash increased correctly

Errors / Issues Found: No errors found

Explanation: This test ensures that **mission completion correctly updates inventory with rewards**.

Test Case 16

Scenario: Assign rewards for repair mission

Modules Involved:

1. MissionPlanningModule
2. MissionRewardsSystem
3. InventoryModule

Steps:

1. Create a repair mission
2. Mark mission as completed
3. Assign rewards

Expected Result:

➔ Spare parts should increase

Actual Result:

➔ Spare parts increased correctly

Errors / Issues Found: No errors found

Explanation: This test ensures that **mission rewards correctly update inventory resources**.

Test Case 17

Scenario: Assign rewards for rescue mission

Modules Involved:

1. MissionPlanningModule
2. MissionRewardsSystem

Steps:

1. Register a crew member
2. Create a rescue mission
3. Mark mission as completed
4. Assign rewards

Expected Result:

- ➔ Reputation should increase
- ➔ Reputation updated correctly

→ Reputation updated correctly

Errors / Issues Found: No errors found

Explanation: This test ensures that **reputation is correctly updated based on mission outcomes**.

Test Case 18

Scenario: Assign rewards to incomplete mission

Modules Involved:

1. MissionRewardsSystem

Steps:

1. Create a mission
2. Do not complete it
3. Attempt to assign rewards

Expected Result:

- ➔ System should return error

Actual Result:

- ➔ Error returned correctly

Errors / Issues Found: No errors found

Explanation: This test ensures that **rewards are only given after successful mission completion**.

Test Case 19

Scenario: Assign rewards for unknown mission type

Modules Involved:

1. MissionRewardsSystem

Steps:

1. Create a mission with unknown type
2. Mark it as completed
3. Assign rewards

Expected Result:

➔ System should return error

Actual Result:

➔ Error returned correctly

Errors / Issues Found: No errors found

Explanation: This test ensures that **invalid mission types are handled safely without affecting system state**.

Test Case 20

Scenario: Assign driver twice to the same race

Modules Involved:

1. RaceManagementModule
2. CrewManagementModule

Steps:

1. Register a crew member "Alex"
2. Assign role "driver"
3. Create a race
4. Assign driver to the race
5. Attempt to assign the same driver again

Expected Result:

- ➔ System should either prevent reassignment or maintain consistency
- ➔ Race should have only one driver assigned

Actual Result:

- ➔ System allowed reassignment of the same driver

Errors / Issues Found: Driver reassignment is allowed without validation

Explanation: This test highlights a **potential design limitation**, where the system allows overwriting driver assignments. It helps identify areas for improving validation logic.

Test Case 21

Scenario: Create duplicate race IDs

Modules Involved:

1. RaceManagementModule

Steps:

1. Create a race with ID 402
2. Attempt to create another race with the same ID

Expected Result:

- ➔ System should reject duplicate race creation
- ➔ Error message should indicate race already exists

Actual Result:

- ➔ First race created successfully
- ➔ Second attempt returned error

Errors / Issues Found: No errors found

Explanation: This test ensures that **race IDs remain unique**, preventing data conflicts and maintaining system integrity.

Test Case 22

Scenario: Upgrade car and then use it in a race

Modules Involved:

1. InventoryModule
2. VehicleUpgradeSystem
3. RaceManagementModule

Steps:

1. Add a car to inventory
2. Add required parts and cash
3. Upgrade the car engine
4. Create a race
5. Assign upgraded car to race

Expected Result:

- ➔ Car should be successfully upgraded
- ➔ Upgraded car should be assignable to race

Actual Result:

- ➔ Car upgraded successfully
- ➔ Car assigned to race correctly

Errors / Issues Found: No errors found

Explanation: This test ensures that **upgraded cars from the inventory can be used seamlessly in races**, validating interaction between upgrade and race modules.

Test Case 23

Scenario: Mission with multiple required roles (all available)

Modules Involved:

1. RegistrationModule
2. CrewManagementModule
3. MissionPlanningModule

Steps:

1. Register two crew members
2. Assign roles "driver" and "mechanic"
3. Create a mission requiring both roles
4. Start the mission

Expected Result:

- ➔ Mission should start successfully
- ➔ All required roles should be validated

Actual Result:

- ➔ Mission completed successfully

Errors / Issues Found: No errors found

Explanation: This test ensures that **missions requiring multiple roles are validated correctly**, confirming proper role-checking logic.

Test Case 24

Scenario: Full system flow (race → mission → upgrade → race)

Modules Involved:

1. RegistrationModule
2. CrewManagementModule
3. InventoryModule
4. RaceManagementModule
5. ResultsModule
6. MissionPlanningModule
7. MissionRewardsSystem
8. VehicleUpgradeSystem

Steps:

1. Register crew members and assign roles
2. Add a car to inventory
3. Create and complete a race
4. Create and complete a mission

5. Assign mission rewards
6. Upgrade the car
7. Create and complete another race

Expected Result:

- ➔ Cash should increase from race rewards
- ➔ Mission rewards should update inventory
- ➔ Car should be upgraded successfully
- ➔ Final race should complete successfully

Actual Result:

- ➔ All operations executed correctly
- ➔ Cash, rewards, and upgrades reflected properly

Errors / Issues Found: No errors found

Explanation: This test validates the **complete system workflow**, ensuring that all modules interact correctly and the system behaves as expected in a real-world scenario.

Test Case 25

Scenario: Car damaged during race and repair mission succeeds when mechanic is available

Modules Involved:

1. RegistrationModule
2. CrewManagementModule
3. InventoryModule
4. RaceManagementModule
5. ResultsModule
6. MissionPlanningModule

Steps:

1. Register crew members "Alex" and "Bob"
2. Assign roles "driver" and "mechanic" respectively
3. Add a car to inventory
4. Create a race and assign driver and car
5. Record race result as "lose" (car gets damaged)

6. Create a repair mission requiring mechanic
7. Start the mission

Expected Result:

→ Car condition should decrease after losing race
→ Repair mission should start successfully since mechanic is available

Actual Result:

→ Car condition reduced correctly
→ Repair mission started successfully

Errors / Issues Found: No errors found

Explanation: This test ensures that **race outcomes affect vehicle condition and that repair operations depend on crew availability**, validating interaction between race, inventory, and mission modules.

Test Case 26

Scenario: Car damaged during race but repair mission fails due to missing mechanic

Modules Involved:

1. RegistrationModule
2. CrewManagementModule
3. InventoryModule
4. RaceManagementModule
5. ResultsModule
6. MissionPlanningModule

Steps:

1. Register a crew member "Alex"
2. Assign role "driver"
3. Add a car to inventory
4. Create a race and assign driver and car
5. Record race result as "lose" (car gets damaged)
6. Create a repair mission requiring mechanic
7. Attempt to start the mission

Expected Result:

→ Car condition should decrease after losing race
→ Mission should fail due to absence of mechanic

Actual Result:

→ Car condition reduced correctly
→ Mission start failed with error

Errors / Issues Found: No errors found

Explanation: This test ensures that **missions enforce role requirements strictly**, preventing operations when required crew members are unavailable.